PCAP / DIGITAL TWIN

Data Center Thermal Simulation Complete Page-by-Page Guide

A teacher-ready explanation of every page, control, displayed value, calculation, chart, and export.

What this project is: a two-dimensional, simulation-based thermal digital twin prototype. It turns server utilization into electrical power and heat, updates temperatures with a native solver, finds hot cells, compares operating scenarios, and measures CPU/OpenMP/CUDA implementations.

What the screenshots show: the actual locally running Streamlit app. The sample case was advanced to step 120; its peak was 27.09 deg C and 24 of 3,072 cells exceeded the 27 deg C hotspot threshold. A 120-step comparison and existing measured benchmark history were also displayed.

Important distinction: all temperatures are calculated. There is no live sensor input, facility calibration, airflow CFD, or cooling electricity model. CUDA source exists, but this machine has no CUDA compiler, so the shown CUDA benchmark rows are failures rather than speed measurements.

Prepared from the project code and a live run on 20 October 2026.

1. How the whole project works

Read this flow first, then use the page screenshots as a map of the interface.

On screen	What it means / what happens
1 - Configuration	Choose grid size, physical coefficients, workload pattern, cooling conditions, hotspot threshold, and solver.
2 - Input fields	At each update index, Python builds a utilization field U for every cell. Scheduled events and persistent selected-cell overrides modify U and cooling h.
3 - Power and heat	For each cell, P = P_idle + U(P_max - P_idle) in watts. Heat source q = P / V in W/m^3. The model assumes all electrical power becomes local heat.
4 - Native update	Four row-major float32 arrays - current temperature, heat, cooling, and cooling-air temperature - are sent to the C++ engine. Sequential CPU, OpenMP, CUDA, and CUDA shared-memory versions use the same discrete equation.
5 - Analytics	Python reads the next full-resolution temperature grid, increments the step counter, and records minimum, mean, maximum, standard deviation, hotspot count, and hotspot percentage.
6 - Dashboard	Overview and Thermal Twin display the current grid and history; Scenario Lab reruns complete strategies; Performance Lab measures implementation time and numerical agreement.

The calculation to explain aloud

rho_cp * dT/dt = k * Laplacian(T) + P/V - h * (T - T_air)

The left side is thermal energy stored per unit volume. Conductivity k spreads heat between neighboring cells. P/V adds server heat. Cooling h removes heat when the cell is warmer than the cooling air. The solver uses an explicit finite-difference step: it reads the previous temperature at a cell and its four neighbors, then writes the next value. Edge cells are held at the ambient boundary temperature.

Time convention: the displayed simulation clock is step count times dt. It is simulated time, not wall-clock runtime. An event scheduled for step 70 changes the inputs used for the update from displayed step 70 to step 71.

2. Shared navigation and chart behavior

On screen	What it means / what happens
Sidebar title and badge	"PCAP / DIGITAL TWIN" identifies the project. "Thermal Twin" is the app title.
Five page labels	Overview, Thermal Twin, Scenario Lab, Performance Lab, and Configuration are a radio navigation group. Exactly one is active; switching pages keeps the current Streamlit session state.
Sidebar status text	"Simulation prototype" warns that displayed temperatures are calculated rather than sensor readings. The next line reports current rows x columns and selected solver.
Plotly toolbar	The small chart toolbar offers PNG download, zoom, pan, zoom in/out, autoscale, reset axes, and fullscreen. These change the view only; they do not change the simulation.
Chart hover	Hovering the heatmap shows row, column, and temperature. Plot legends label plotted series. Heatmap display may be downsampled above 120 cells on an axis; metrics and CSV still use the full numerical grid.
Number-input +/- buttons	These adjust the adjacent number field. Values affect the model only when the relevant Apply, Run, or Schedule button is pressed.
Table toolbar	Streamlit data tables may offer show/hide columns, search, direct CSV download, and fullscreen. The page-level export button is a separate convenience control.

3. Overview - actual screen



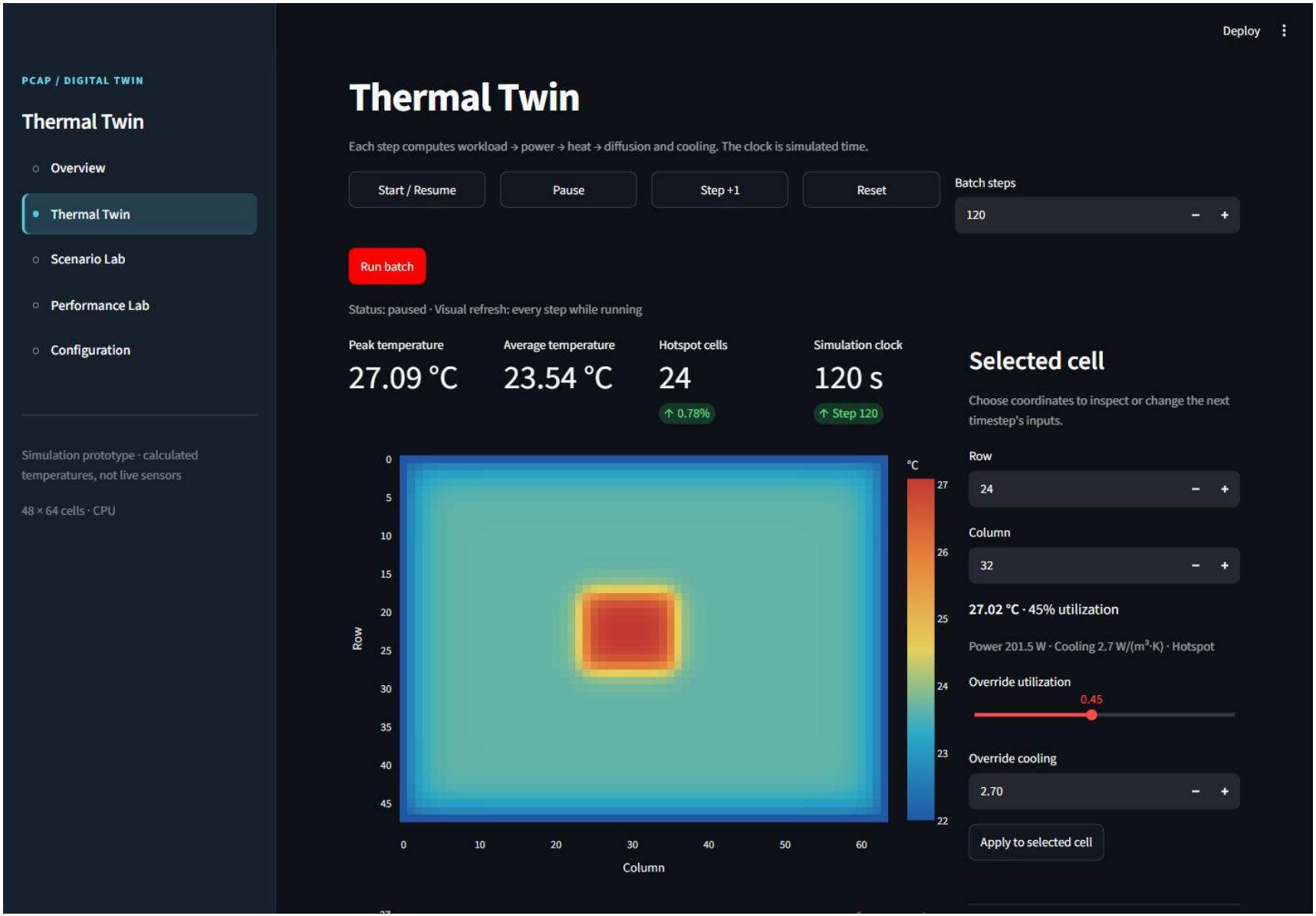
Live screenshot after 120 sample steps. The colored center shows the built-in localized cooling failure.

3. Overview - every displayed item

Purpose: a read-only summary of the current simulation state. Use Thermal Twin or Configuration to change it.

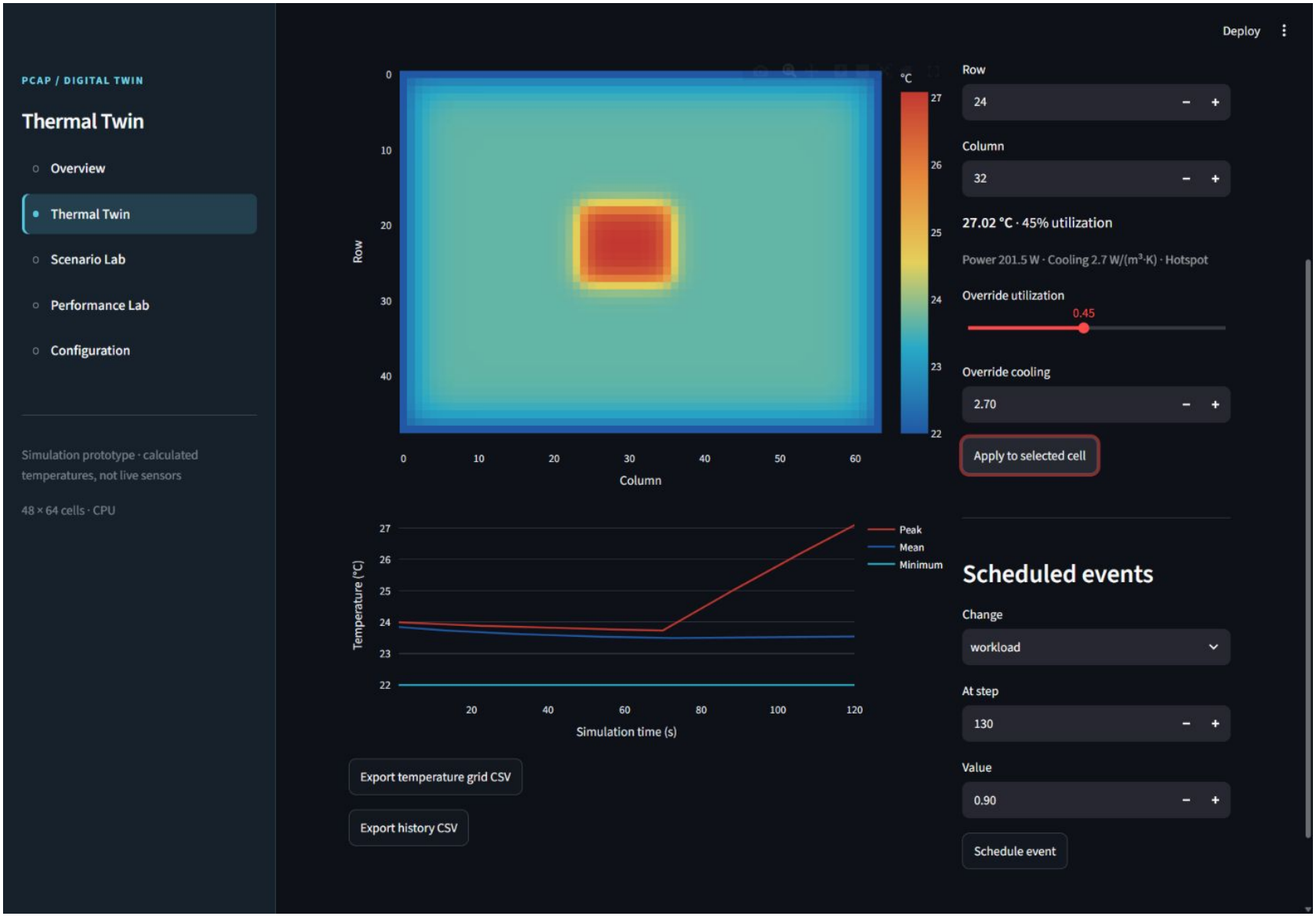
On screen	What it means / what happens
Page eyebrow/title/caption	"Engineering overview" and "Data center thermal state" identify the summary. The caption states the model is 2D transient, uses a fixed ambient boundary, and has simulated server load.
Peak temperature	Maximum temperature among all rows x columns of the current numerical grid. At step 120 in the screenshot it is 27.09 deg C.
Average temperature	Arithmetic mean of all cells, including boundary cells. The shown value is 23.54 deg C.
Hotspot cells + percentage	Count of cells strictly warmer than the configured threshold; percentage = 100 x count / (rows x columns). The sample has 24 / 3,072 = 0.78%.
Simulation clock + step	Clock = completed steps x configured timestep. At dt = 1 s, step 120 means 120 s of simulated time.
Sample-loaded info message	Shown only when step = 0. It points the user to Thermal Twin for advancing or Configuration for editing.
Heatmap	Each colored tile is a modeled temperature. Horizontal axis is column; vertical axis is row; row 0 is at the top. The color bar is deg C, blue cooler to red hotter. The edge stays at ambient.
Temperature history	Three lines use saved per-step summaries: Peak = max, Mean = average, Minimum = min. Horizontal axis is simulation time in seconds; vertical axis is deg C. No history appears before the first step.
Threshold	Current hotspot temperature limit, 27 deg C by default. Equality does not count as a hotspot.
First crossing	First recorded completed step whose maximum is strictly above threshold. The screenshot shows step 119.
Time above	Sum of dt for recorded steps that contain at least one hotspot. Here it is 2 s, not the accumulated temperature excess.
Recovery after restoration	If a restore_cooling event occurs after a hotspot, the first subsequent zero-hotspot record gives elapsed recovery time. Otherwise the page says "not measured".
Workload / Solver	Shows the current workload pattern and configured native implementation. The screenshot uses steady workload and CPU.

4. Thermal Twin - controls and heatmap



Live screenshot at step 120. The selected center cell is above the threshold after cooling drops to 2.7 W/(m^3 K).

4. Thermal Twin - history and events



The lower portion shows the temperature time series, CSV exports, selected-cell override, and scheduled-event form.

4. Thermal Twin - playback and live field

Purpose: advance, inspect, and intervene in one stateful simulation.

On screen	What it means / what happens
Start / Resume	Sets running status. A Streamlit fragment refreshes about every 600 ms and advances one model step on each refresh. It continues until paused or an error occurs.
Pause	Stops automatic stepping; it preserves temperature, history, events, and cell overrides.
Step +1	Runs exactly one native update immediately. If automatic running is also enabled, subsequent fragment refreshes can continue advancing.
Reset	Stops automatic running; recreates the initial field; resets step to 0; clears history and selected-cell overrides. The current configuration and its scheduled events remain.
Batch steps / Run batch	Choose 1 to 500 updates, then click Run batch. This advances that many steps without requiring the live refresh loop; it does not mean seconds unless dt = 1 s.
Status / Visual refresh	Shows running or paused and explains that visual updates occur each step while automatic running is enabled. This is UI status, not solver benchmark timing.
Four KPIs	Peak, average, hotspot cells with percentage, and simulation clock with completed step use the same definitions as Overview and update from the current full grid.
Heatmap and history	The heatmap uses the current temperature field. The timeline plots recorded peak, mean, and minimum at every completed step.
Export temperature grid CSV	Downloads the current full-resolution temperature array without row or column headers. Position in the matrix is the grid coordinate.
Export history CSV	Downloads one record per completed step with step, time_s, dt, min, mean, max, std, hotspot_count, and hotspot_pct.

Why the sample center becomes hot

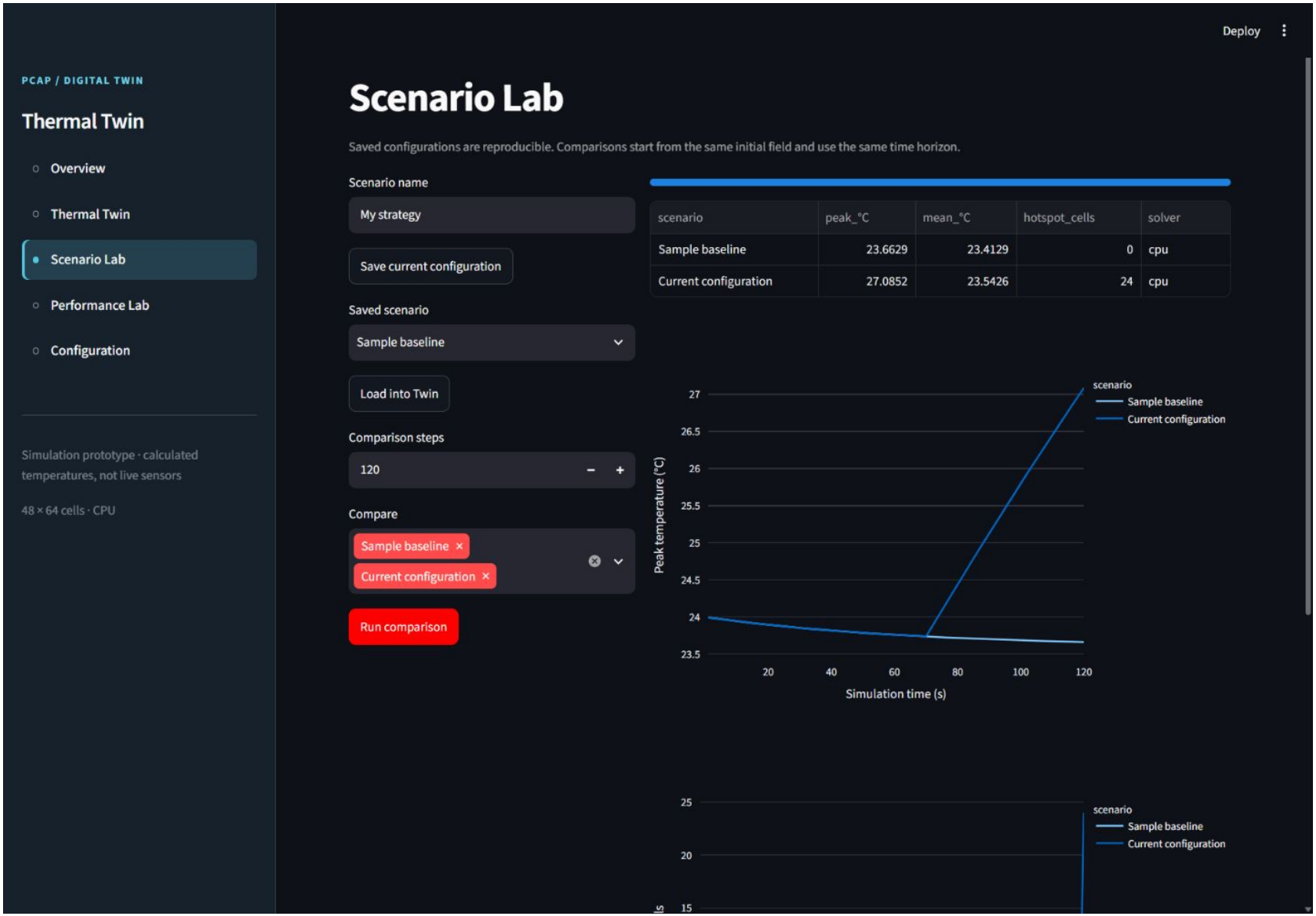
Default cooling is 18 W/(m^3 K). The failure begins at update index 70 in the rectangle starting at row 18, column 24, size 10 x 12. Its factor 0.15 makes local cooling 18 x 0.15 = 2.7 W/(m^3 K). With steady utilization 0.45, local server power is 80 + 0.45 x (350 - 80) = 201.5 W and heat is 201.5 / 2 = 100.75 W/m^3. The weaker cooling lets the center rise above 27 deg C.

4. Thermal Twin - selected cell and scheduled events

On screen	What it means / what happens
Row / Column	Zero-based coordinates for the cell to inspect. The valid ranges are 0 to rows-1 and 0 to columns-1. These inputs select an inspection point; they do not change temperature themselves.
Selected temperature	The current stored temperature at those coordinates, in deg C. It comes from the last completed native update.
Utilization percentage	The utilization field that would be used for the next update at the selected cell, including current pattern, events, and any saved override.
Power / Cooling / Hotspot	Power is P_idle + U(P_max-P_idle) in W. Cooling is local h in W/(m^3 K). Hotspot means current temperature is strictly above threshold; "Below threshold" means it is not.
Override utilization	Slider from 0 to 1. It replaces utilization for this cell only, affecting its future power and heat source. It does not set temperature directly.
Override cooling	Nonnegative local cooling coefficient in W/(m^3 K). It changes this cell only. A larger value removes more modeled heat when T exceeds cooling-air temperature.
Apply to selected cell	Stores both override values for that coordinate. They take effect in the next step, persist across later steps, and are cleared by Reset or loading/applying a new configuration.
Change = workload	Schedules a global utilization value in [0,1] for that step and later updates. Its Value is utilization, not watts.
Change = cooling	Schedules a global cooling coefficient (nonnegative W/(m^3 K)) for that step and later updates.
Change = redistribute	At each applicable update, halves load in the configured localized region and distributes the removed utilization across the other cells. Value is disabled. The final utilization is clipped to [0,1], so exact conservation can be lost at the cap.
Change = restore_cooling	Restores the global base cooling coefficient after the built-in failure is applied. Value is disabled. A later cooling event or cell override can still change an applicable cell.
At step / Value	At step is a zero-based future or current update index; event effects start before the update from that index to the next completed step. Value is enabled only for workload and cooling.
Schedule event / event table	Validates and appends the event to current configuration and simulation. The table lists scheduled step, kind, and value when present. Events are applied in step order on every later field generation.

Useful distinction for the teacher: a workload or cooling intervention changes the inputs to the heat equation. It does not directly edit the existing temperature field. Temperature changes only after subsequent solver steps.

5. Scenario Lab - comparison result



A real 120-step comparison: no-failure sample baseline versus the current sample with cooling failure.

5. Scenario Lab - hotspot trajectories



The second plot compares hotspot count across the same simulation horizon.

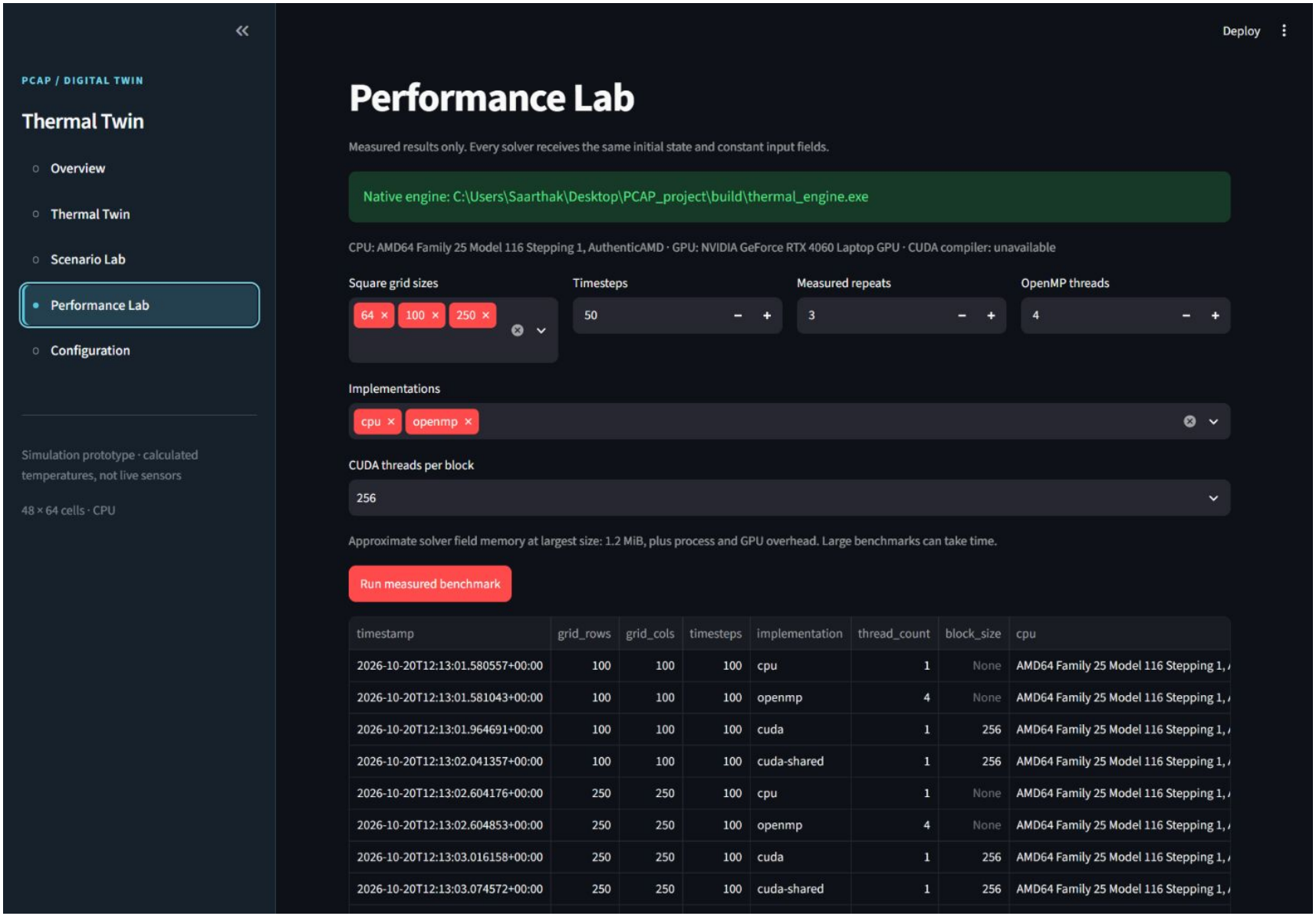
5. Scenario Lab - every control and result

Purpose: save reproducible settings and compare thermal outcomes from a common starting setup.

On screen	What it means / what happens
Scenario name	Human-readable name for a saved configuration. Saving turns it into a lowercase filename slug in data/scenarios; the JSON stores both name and configuration. Saving the same slug overwrites that scenario file.
Save current configuration	Validates and saves current settings to JSON. It saves configuration and scheduled events, not the already-evolved temperature or history.
Saved scenario	Selects Sample baseline or a previously saved JSON scenario. Sample baseline is default settings with the built-in cooling failure postponed far beyond normal runs.
Load into Twin	Replaces the current configuration with the selected one, creates a fresh simulation at step 0, and stops automatic running. This is a load action, not a comparison run.
Comparison steps	Number of updates each selected scenario receives, 1 to 500. Each run starts from its own configured initial field; the interface checks that key initial/model parameters match.
Compare	Multi-select two or more of Sample baseline, Current configuration, and saved scenarios. The Run button is disabled until at least two are selected.
Run comparison / progress	Independently creates and advances each selected scenario for the chosen horizon. A progress bar tracks completed scenarios. Failed scenarios display an error and are omitted from successful results.
Fairness check	Rows, columns, initial temperature, ambient, dx, dy, rho_cp, conductivity, cell volume, and hotspot threshold must match the first scenario. This keeps the starting field and thermal model comparable while strategies can vary.
Results table: scenario	Name of each successfully run strategy.
Results table: peak_deg C	Final maximum grid temperature for that scenario.
Results table: mean_deg C	Final average grid temperature.
Results table: hotspot_cells	Final count strictly above the common threshold.
Results table: solver	Native solver used for that scenario. It is an execution choice, not a thermal strategy metric.
Peak-temperature chart	One line per scenario: recorded maximum at each simulation time in seconds. In the sample, the current case rises after the cooling failure while baseline remains around 23.66 deg C.
Hotspot-count chart	One line per scenario: number of hotspot cells at each time. The sample comparison ends at 24 for current configuration and 0 for baseline.
Export comparison CSV	Downloads the final results table only, with scenario, peak, mean, hotspot count, and solver. It does not include per-step chart traces.

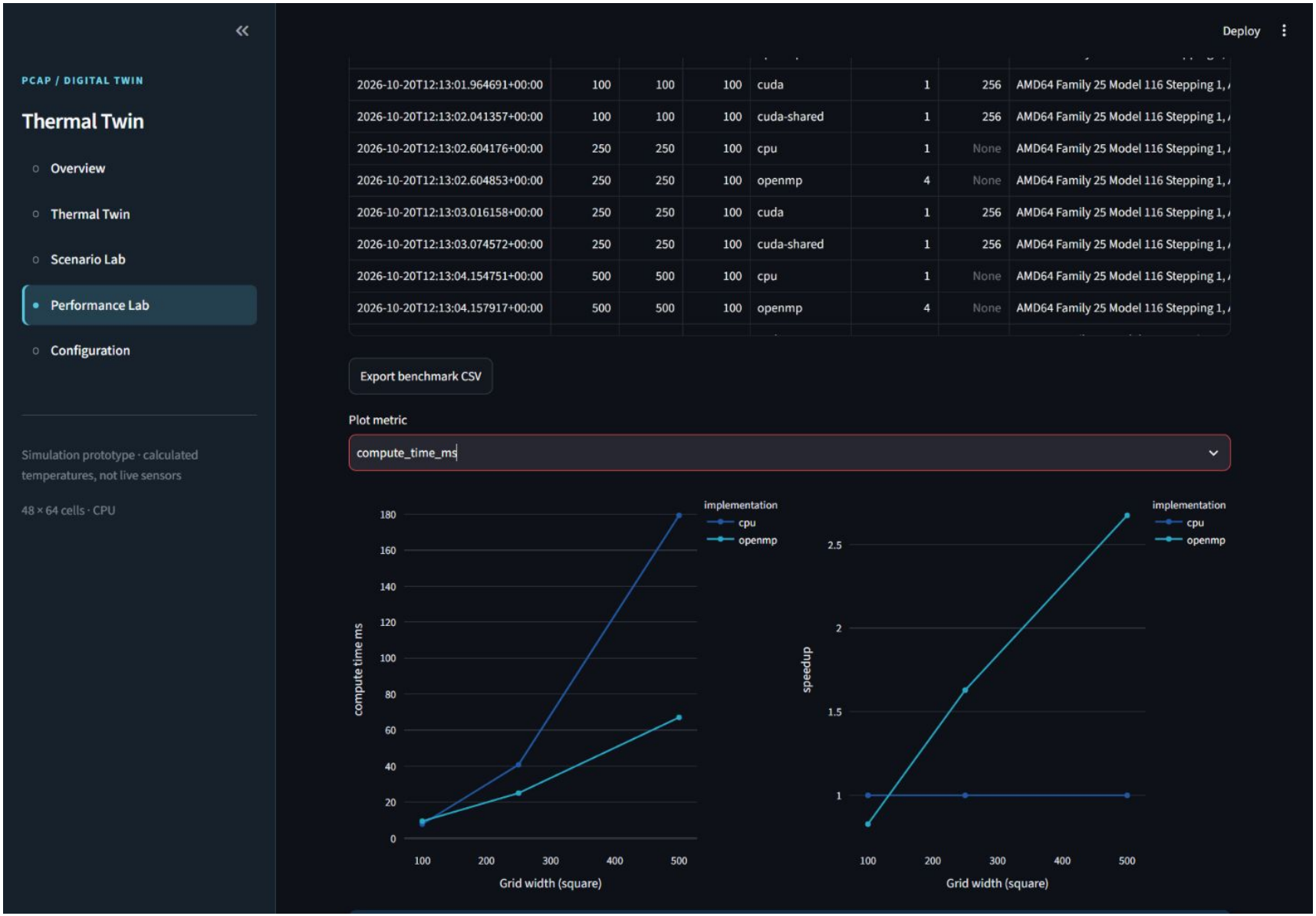
How to interpret the shown experiment: Sample baseline ends at 23.6629 deg C peak and 0 hotspots; current configuration ends at 27.0852 deg C peak and 24 hotspots. The controlled difference is the built-in cooling failure. These are model outputs, not measured facility temperatures.

6. Performance Lab - benchmark setup



The native engine is detected. The saved results table includes CPU/OpenMP measurements and CUDA failure rows.

6. Performance Lab - timing charts



Compute time and speedup from saved measured results. Only successful implementations are plotted.

6. Performance Lab - setup and experimental method

Purpose: compare implementations of the same numerical update, not thermal operating strategies.

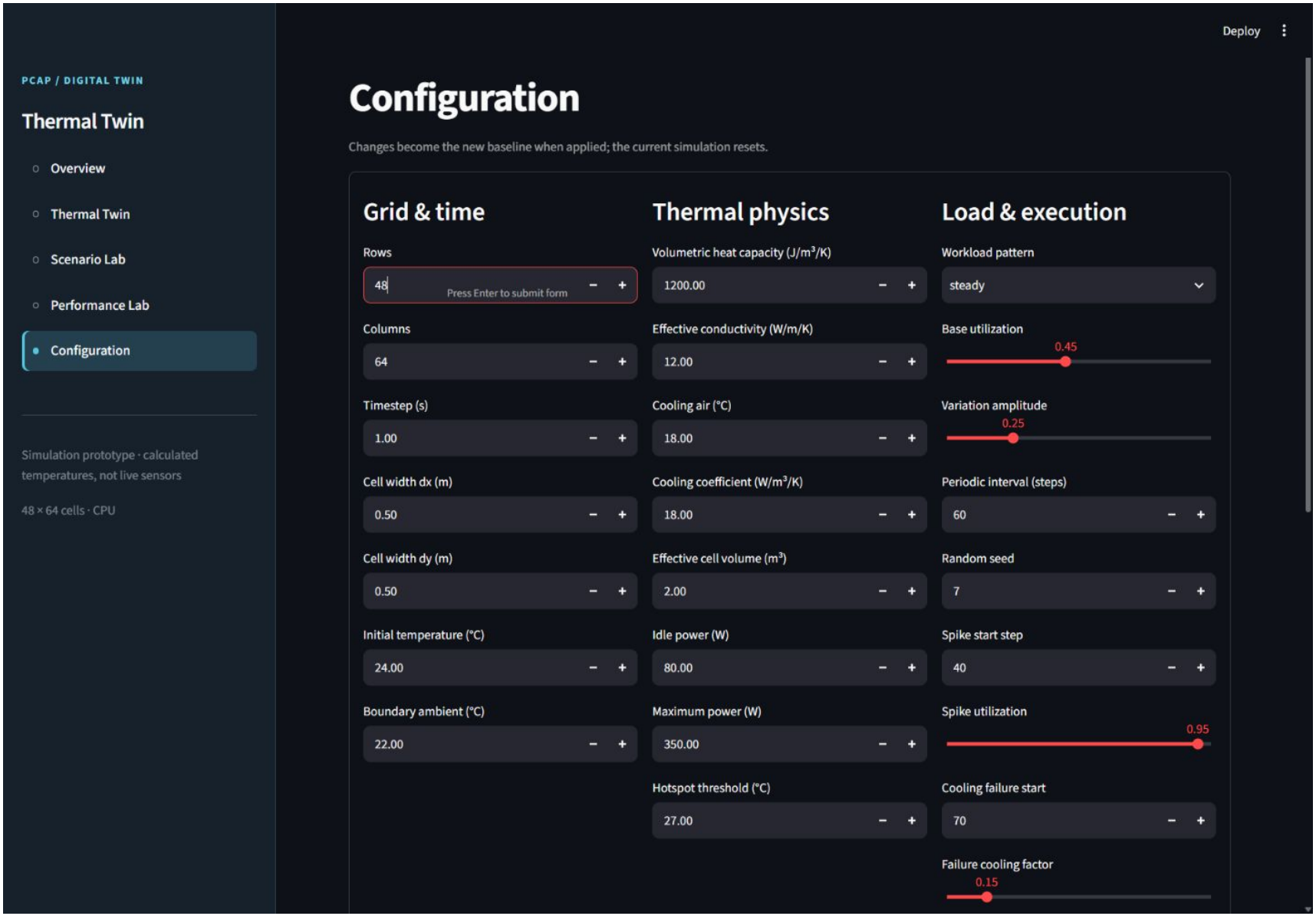
On screen	What it means / what happens
Native engine status	Displays the actual executable path or an error if the C++ engine is missing.
CPU / GPU / CUDA compiler	Reports local processor string, detected GPU name, and nvcc path or "unavailable". Detecting a GPU driver does not mean the CUDA solver was built.
Square grid sizes	Multi-select from 32, 64, 100, 250, 500, 1000, 2000, 4000. Each chosen N becomes an N x N benchmark grid.
Timesteps	Number of native updates per measured run. Benchmark input fields are held constant during these steps.
Measured repeats	Number of measured repetitions per implementation and size, 1 to 20. The table reports medians to reduce timing noise.
OpenMP threads	Requested CPU thread count, 1 to 128, for OpenMP rows. It does not control sequential CPU or CUDA thread blocks.
Implementations	Choose cpu, openmp, cuda (global memory), and/or cuda-shared (tiled shared memory). CPU reference is still run even if cpu is not selected for the visible results.
CUDA threads per block	Selects 64, 128, 256, 512, or 1024 threads per CUDA block for GPU runs. It has no effect on CPU/OpenMP results.
Memory estimate	Approximate 5 float32 fields x largest N^2 / 2^20 MiB. It excludes extra process and GPU allocation overhead.
Run measured benchmark	For each size: copy current physical settings, force steady load, disable failure during run, clear events, use a quarter-grid region, and give all implementations identical initial state and heat/cooling/air fields. The UI progress bar counts size-implementation combinations.
Warm-up and baseline	Sequential CPU is the numerical reference and timing baseline. Non-CPU implementations get one warm-up step before measured repeats.
Results persistence	Every new run is saved as a timestamped CSV under data/benchmark_results. On opening the page, recent saved CSV files are loaded; displayed rows may come from earlier sessions.
Export benchmark CSV	Downloads all currently displayed result rows.
Plot metric	Selects compute_time_ms, end_to_end_time_ms, speedup, throughput, max_absolute_error, kernel_time_ms, or transfer_time_ms for the left chart. The right chart always shows speedup.
OpenMP thread chart	Appears only if successful OpenMP results have more than one distinct thread count; it plots compute time versus thread count by grid size.
CUDA information message	Explains that a missing toolkit or incompatible build produces failed CUDA rows. No CPU timing is silently substituted as a GPU timing.

6. Performance Lab - every result column and formula

On screen	What it means / what happens
timestamp	UTC timestamp when that result row was created.
grid_rows / grid_cols	Dimensions of the square benchmark grid, both equal to selected N.
timesteps	Number of native updates in each measured repeat.
implementation	Solver backend: cpu, openmp, cuda, or cuda-shared.
thread_count	Requested OpenMP threads for OpenMP rows; 1 is recorded for other implementations.
block_size	Requested CUDA block size for CUDA rows; empty for CPU and OpenMP.
cpu / gpu_name	Detected CPU string; GPU device name is populated only for a successful CUDA result.
success / error	Whether the implementation completed. A failed row retains its exception text and has no fabricated timing or speedup.
compute_time_ms	Median native-engine computation interval. For GPU this native interval includes GPU setup/transfer work; kernel_time_ms isolates kernel execution.
variability_ms	Population standard deviation of repeated compute-time measurements. Zero with one repeat does not mean stable performance.
end_to_end_time_ms	Median Python wall time around native process launch, binary input/output, and execution. This is especially relevant to interactive one-step calls.
kernel_time_ms	Median measured CUDA kernel time. Zero for CPU/OpenMP.
transfer_time_ms	Median CUDA host-device transfer time. Zero for CPU/OpenMP.
speedup	Median sequential CPU compute time divided by this row's median compute time at the same grid size. CPU baseline is 1. Values above 1 are faster; below 1 are slower.
throughput	Grid rows x grid columns x timesteps divided by compute time in seconds, reported as cell-updates per second.
max_absolute_error	Largest absolute difference between final implementation grid and sequential CPU reference, in deg C.
mean_absolute_error	Average absolute final-grid difference from the CPU reference, in deg C.
within_tolerance	True when max absolute error <= 0.001 deg C. This checks numerical agreement only, not agreement with a real facility.
efficiency	For OpenMP, speedup / OpenMP thread count. Empty for other backends.

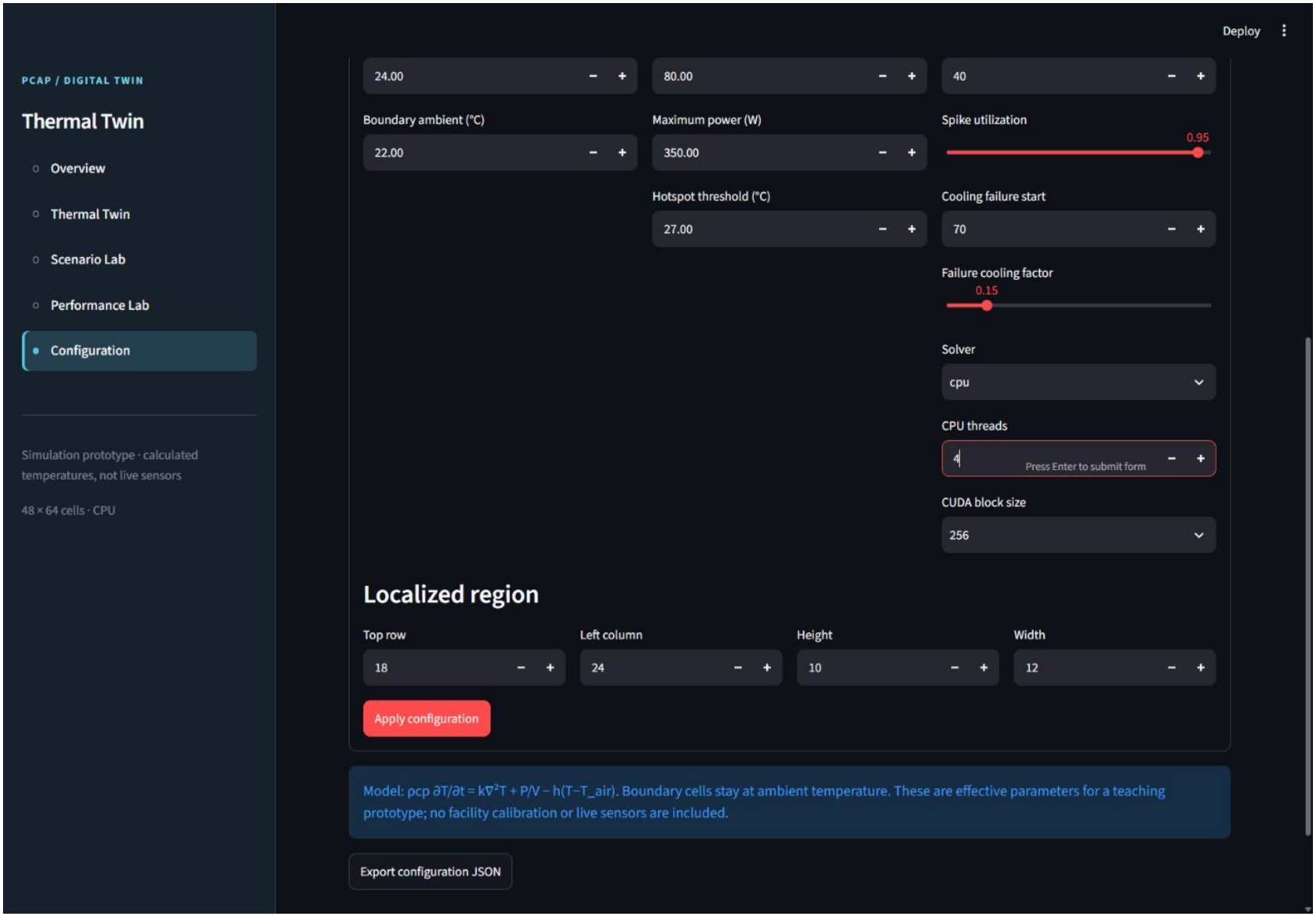
Observed environment: this computer reports an NVIDIA GeForce RTX 4060 Laptop GPU, but nvcc is unavailable. Saved CPU/OpenMP rows are measured; saved CUDA and CUDA-shared rows say CUDA support was not built. Do not claim a GPU speedup from those failed rows.

7. Configuration - grid and physics



The top form contains grid/time, thermal physics, and the first workload controls, with sample defaults visible.

7. Configuration - execution controls



A second live view covers the lower workload controls, solver settings, and localized region.

7. Configuration - grid, time, and thermal physics

Purpose: define the model parameters. Apply configuration validates them and resets the live simulation.

On screen	What it means / what happens
Rows / Columns	Number of temperature cells vertically and horizontally. Default is 48 x 64 = 3,072 cells. Each must be at least 3 in the form; total must not exceed 100 million.
Timestep (s)	dt, simulated seconds per numerical update. Default 1 s. Too large a dt relative to diffusion/cooling is rejected by stability validation.
Cell width dx (m)	Spatial spacing used in the row-neighbor second derivative. Default 0.5 m. This is an effective model distance.
Cell width dy (m)	Spatial spacing used in the column-neighbor second derivative. Default 0.5 m.
Initial temperature (deg C)	Interior cells start at this temperature when simulation is created or reset. Default 24 deg C.
Boundary ambient (deg C)	All edge cells are pinned to this value at initialization and every solver step. Default 22 deg C; it is different from cooling-air temperature.
Volumetric heat capacity	rho_cp in J/(m^3 K), default 1,200. It controls how much heat input is needed for a temperature change; larger values slow temperature response.
Effective conductivity	k in W/(m K), default 12. It spreads heat by the neighboring-cell Laplacian. This is an effective coefficient, not a detailed airflow model.
Cooling air (deg C)	T_air in the cooling-loss term h(T - T_air), default 18 deg C. The app uses a uniform cooling-air field.
Cooling coefficient	Base h in W/(m^3 K), default 18. Larger h means stronger local heat removal for a given cell-air temperature difference.
Effective cell volume (m^3)	V in q=P/V, default 2 m^3. For fixed watts, a smaller V makes a larger volumetric heat source.
Idle power (W)	Electrical power when utilization is zero, default 80 W per modeled cell.
Maximum power (W)	Electrical power when utilization is one, default 350 W. Validation requires it to be at least idle power.
Hotspot threshold (deg C)	A user-selected study threshold, default 27 deg C. A cell is hot only when T is strictly greater than it.

7. Configuration - workload, failure, and execution

On screen	What it means / what happens
Workload pattern: steady	Every cell uses Base utilization, except subsequent scheduled events or cell overrides. This is the default.
Pattern: random	For each update index, seeded random values vary cell utilization around base by plus/minus Variation amplitude, then clip to [0,1]. Seed + step makes pause/resume reproducible.
Pattern: periodic	All cells follow base + amplitude x sin(2*pi*step/period), clipped to [0,1].
Pattern: spike	All cells use base until Spike start step, then use Spike utilization.
Pattern: localized	Only the configured rectangle uses Spike utilization; the rest use Base utilization.
Pattern: moving	A rectangle at the configured top row moves one column per update, wrapping within the grid; it uses Spike utilization.
Base utilization	Fraction of maximum modeled load, 0 to 1, default 0.45. With default powers it gives 201.5 W per cell.
Variation amplitude	Maximum deviation around base for random/periodic patterns, default 0.25. It has no effect on steady/spike/localized/moving.
Periodic interval (steps)	Period of the sinusoidal workload in number of update indices, default 60; used only by periodic pattern.
Random seed	Nonnegative integer that makes random workload reproducible, default 7; used only by random pattern.
Spike start step	Zero-based update index when global spike starts, default 40; used only by spike pattern.
Spike utilization	Load level in spike, localized, and moving patterns, default 0.95.
Cooling failure start	Zero-based update index from which base cooling in the localized region is multiplied by Failure cooling factor. Default 70. This can act with any workload pattern.
Failure cooling factor	Fraction of base cooling retained in that region after failure, default 0.15. Example: 18 x 0.15 = 2.7 W/(m^3 K).
Solver	Choose sequential cpu, openmp, cuda, or cuda-shared for live updates. CUDA requires a CUDA-enabled native build; an unavailable backend raises an error.
CPU threads	Requested OpenMP threads, default 4. It does not parallelize the sequential cpu solver.
CUDA block size	CUDA threads per block, default 256; options are 64, 128, 256, 512, 1024. Used only by CUDA solvers.

7. Configuration - region, validation, and export

On screen	What it means / what happens
Top row / Left column	Zero-based origin of the special rectangle. Defaults row 18, column 24.
Height / Width	Size of that rectangle, default 10 rows x 12 columns. The form prevents it from extending outside the grid.
Why the region matters	It identifies where localized and moving high load is placed, where built-in cooling failure applies, and which load is reduced by redistribute events.
Apply configuration	Builds a candidate from all form fields, validates it, replaces current configuration, creates a fresh simulation, stops running, and returns to step 0. Existing scheduled events are retained in the candidate unless changed elsewhere.
Validation rules	Positive dt/dx/dy/rho_cp/volume; nonnegative conductivity/cooling/powers; utilization and failure factor within [0,1]; valid pattern/solver; region within grid; max power >= idle; and explicit-step stability.
Stability expression	2*k*dt/rho_cp*(1/dx^2 + 1/dy^2) + h_max*dt/rho_cp <= 1. The current maximum cooling field is rechecked during steps, so an event or cell override can also trigger an instability error.
Model info panel	Restates the heat equation and warns that coefficients are effective teaching parameters, boundary cells stay ambient, and there is no facility calibration or live sensors.
Export configuration JSON	Downloads the current configuration as thermal-config.json, including fields such as default steps and event list. The form itself does not expose the stored steps value; live batch and scenario comparison have their own step-count controls.

Important temperature distinction: Boundary ambient fixes outer-grid cells at 22 deg C by default. Cooling air is 18 deg C inside the cooling equation. The two fields have different roles and may be set independently.

8. A clear demonstration for your teacher

On screen	What it means / what happens
1 - Start on Overview	Explain the four KPIs, heatmap axes/color, three history lines, and threshold. State that all values come from the numerical grid.
2 - Open Configuration	Show default 48 x 64 grid, dt = 1 s, steady 45% utilization, power range 80-350 W, base cooling 18, failure step 70 and factor 0.15. Point out the 10 x 12 region.
3 - Open Thermal Twin	Run a 120-step batch. Show clock = 120 s, peak near 27.09 deg C, 24 hotspots, and the central red region. The selected cell demonstrates cooling falling to 2.7 while utilization remains 45%.
4 - Explain an intervention	Choose a cell override or schedule restore_cooling for a later step. Explain that changes affect future inputs, then advance the solver to observe the temperature response.
5 - Open Scenario Lab	Compare Sample baseline with Current configuration for 120 steps. Explain why the baseline remains under 27 deg C and the failure case develops 24 hotspots. Show both charts and the export.
6 - Open Performance Lab	Point to measured CPU/OpenMP rows and speedup formula. Explain that CUDA rows failed because this native build lacks CUDA, so no GPU performance conclusion is available here.
7 - Close with limits	Say this is a reproducible engineering teaching model. It demonstrates methods and relative scenario effects, but is not a calibrated predictor of a real data center.

Questions you may be asked

On screen	What it means / what happens
Why is the edge blue?	The model pins every boundary cell to ambient temperature, 22 deg C by default.
Why does the center heat after step 70?	The configured rectangle keeps only 15% of its base cooling from update index 70 onward, while workload and heat generation continue.
Is the clock real time?	No. It is completed steps multiplied by the configured timestep. Runtime on the computer can be different.
What is a hotspot?	Any cell strictly above the configured threshold. The count and percentage use the full numerical grid.
Does GPU detection prove speedup?	No. A CUDA compiler and successful native CUDA run are needed; failed rows contain no performance values.

9. Glossary, evidence, and honest limits

On screen	What it means / what happens
Cell / grid	An effective two-dimensional thermal region and its rows x columns collection. It does not map to a measured rack unless separately calibrated.
Utilization U	A dimensionless load fraction from 0 to 1; it determines modeled electrical power.
Heat source q	Modeled watts divided by effective cell volume, in W/m^3.
Diffusion	Neighbor-to-neighbor temperature spreading represented by conductivity times the discrete Laplacian.
Cooling h	Linear heat-removal coefficient. Its effect depends on local T minus cooling-air temperature.
Native solver	Compiled C++ executable receiving binary arrays. OpenMP adds CPU parallelism; optional CUDA backends run on a compatible GPU build.
Digital twin prototype	A computational representation for controlled experiments. Here it has no live telemetry or facility calibration.

Code and screenshot basis

This guide was checked against the Streamlit page definitions in python/app.py; field generation and validation in python/model.py; stepping in python/simulation_service.py; analytics in python/analytics.py; scenario persistence in python/scenario_service.py; benchmark definitions in python/benchmark_service.py; charts in python/visualization.py; native bridge in python/engine_bridge.py; and the C++ CPU/main engine. Defaults come from config/default_config.json. Screenshots were captured from the locally running app.

What the model cannot prove

The calculation omits detailed airflow, vertical stratification, calibrated rack geometry, sensor assimilation, and cooling electrical consumption. Its threshold is user-selected. Scenario Lab reports thermal outcomes, not an energy optimum. Agreement between two implementations does not establish physical accuracy. Benchmark times depend on hardware and background load.